Audit report — Visionia Electronics

Run #2105 · version 1.0.3 · B1072-24246 · M. Ismail H · 7 Nov 2026 13:45 UTC · engine claude-sonnet-5-5

72 / 100

Threshold 75 · no failing check

Rejected — below threshold

Below threshold or a failing check — fix the items marked must-fix and upload again.

Security	Structure	Completeness	Dependencies	Build
73	72	80	72	72

What to do

0 checks marked must-fix. Work through the findings check by check — each names the file and line, what we found, what we expected and what to change. Then upload the corrected archive as a new version; the audit is free every time and the reviewer sees every attempt.

1 · REQUIREMENT CHECKLIST — 12/15 MET

MUST HAVE · 8/10

✓ **Complete source for every component** PRESENT
Source present for web/API (app/, actions/, lib/), admin (app/admin/, components/admin/, lib/admin.ts), and payments; no compiled-only or obfuscated parts in the tree. No mobile app is declared.

✓ **No real credentials in the archive** PRESENT
No real .env in tree. .env.example uses placeholders (replace-with-*, USER:PASSWORD@HOST). The 12 secret-scan hits are code identifiers or placeholders, not credentials.

✓ **Builds from a clean machine** PRESENT
HowToSetup.md exists; package.json has build script with prisma generate; live Vercel deployment indicates the project builds. Full HowToSetup contents were truncated, so the clean-machine path is not fully verified.

✓ **Database script or migrations** PRESENT
database/schema.sql, database/seed.sql, prisma/migrations/ (7 migrations incl. init), prisma/schema.prisma, and prisma/seed.ts all present.

½ **Authentication and roles on every mutating endpoint** PARTLY
app/admin/layout.tsx and lib/admin.ts suggest role guards, and registerUser is public by design. actions/admin.ts, actions/cart.ts, actions/newsletter.ts and the admin route handlers are not shown, so per-endpoint auth and role checks cannot be confirmed. Duplicate payment routes exist (api/payment and api/payments).
Do this: Show or add a requireRole('ADMIN') check on every mutating action in actions/admin.ts and admin routes; confirm each cart/newsletter mutation requires a session; consolidate the duplicate api/payment and api/payments trees.

½ **Working live demo** PARTLY
Demo URL https://visionia-electronics.vercel.app/ returns HTTP 200 with the expected title. No demo credentials are provided in the digest, so login with them cannot be confirmed.
Do this: Provide demo account credentials (customer and admin) in the submission and verify that sign-in succeeds on the live demo.

✓ **`README.md`** PRESENT
README.md

✓ **`HowToSetup.md`** PRESENT
HowToSetup.md

✓ **`.env.example`** PRESENT
.env.example

✓ **`database/schema.sql`** PRESENT
database/schema.sql

NICE TO HAVE · 4/5

½ **Pinned dependencies without critical CVEs** PARTLY
pnpm-lock.yaml is committed and packages are recent (next 16.4.3, react 19.3.0). No audit output or evidence of vulnerability review is included, and next-auth is on a beta channel.
Do this: Run pnpm audit, upgrade flagged packages, and include the audit result or a note in CHANGELOG.md; consider moving off the next-auth beta.

✓ **Documentation set** PRESENT
README.md, HowToSetup.md, AGENTS.md (accepted equivalent), and CHANGELOG.md are all at the ZIP root.

✓ **`CLAUDE.md (or AGENTS.md)`** PRESENT
AGENTS.md (equivalent file — accepted)

✓ **`CHANGELOG.md`** PRESENT

Audit report — Visionia Electronics

Run #2105 · version 1.0.3 · B1072-24246 · M. Ismail H · 7 Nov 2026 13:45 UTC · engine claude-sonnet-5-5

CHANGELOG.md

☑ **database/seed.sql** PRESENT
database/seed.sql

2 · QUALITY CHECKS — SCORE, FINDINGS AND FIXES

1 · Secrets & credentials — PASS · 93/100

What we look for: Every file is scanned for API keys (AWS, Stripe, Google, GitHub, Slack…), private keys, JWT signing secrets, SMTP passwords and database connection strings with real credentials. Example and placeholder values are fine.

No real credentials found; only clear placeholders in env example and docs
All regex candidates were reviewed. The .env.example and HowToSetup.md hits are obvious placeholders, and the code hits in auth, actions and validations are variable names or env lookups, not secrets. No committed .env, keys, or connection strings with real credentials appear in the visible files, though some files were truncated in the digest.

1.1 `.env.example:8`
Found: AUTH_SECRET set to an obvious placeholder (replace-with-a-random-secret).
Expected: Placeholder value, which is what is present.
Do this: No change required. Optionally document generating the value with `openssl rand -base64 32` and make startup fail if it is unchanged.

1.2 `.env.example:24`
Found: SSLCOMMERZ_STORE_PASSWORD uses a placeholder value, not a real merchant password.
Expected: Placeholder plus env var, as present.
Do this: No change required.

1.3 `HowToSetup.md:154`
Found: Documentation shows a sandbox store password as an obvious placeholder.
Expected: Placeholder in docs, as present.
Do this: No change required.

1.4 `lib/auth.ts:89`
Found: Auth secret read from process.env.AUTH_SECRET; no hardcoded fallback.
Expected: Environment variable with no default, as present.
Do this: No change required. Consider failing fast at boot if AUTH_SECRET is missing in production.

1.5 `lib/payments/sslcommerz.ts:54`
Found: Store password read from the environment, not hardcoded.
Expected: Environment variable, as present.
Do this: No change required.

2 · Dependency vulnerabilities — WARNING · 72/100

What we look for: Every manifest and lockfile (package.json, packages.lock.json, composer.lock, pubspec.lock, requirements.txt…). Versions with publicly known critical vulnerabilities, unpinned ranges, pre-release builds, and packages that have been abandoned or renamed.

Lockfile present, but beta auth, caret ranges and an unverifiable patched SDK reduce confidence
A pnpm-lock.yaml is committed and the core framework packages (next, react, prisma) are pinned exactly, so there is no missing lockfile. I cannot confirm any specific critical CVE from the digest, so nothing is a fail. Warnings: next-auth is a pre-release beta on the authentication path, most dependencies use caret ranges, and the payment SDK sslcommerz-lts is a small, rarely updated package that is also locally patched.

2.1 `package.json:31`
Found: "next-auth": "^5.0.0-beta.29" is a pre-release build that handles login and sessions, and the caret range can drift to newer betas.
Expected: A pinned exact version of a stable release, or a pinned beta that has been reviewed.
Do this: Pin it exactly with `pnpm add next-auth@5.[hidden] --save-exact`, check the Auth.js security advisories, and move to the stable v5 release once available.

2.2 `package.json:35`
Found: "sslcommerz-lts": "1.2.0" is a small third-party payment SDK, locally patched via patches/sslcommerz-lts@1.2.0.patch. It is likely unmaintained and unaudited.
Expected: A maintained payment dependency, or documented review of the patch and the SDK's transitive dependencies.
Do this: Run `pnpm audit --prod`, review the patch, and consider calling the SSLCommerz REST API directly if the SDK stays stale.

2.3 `package.json:14`
Found: Most runtime dependencies use caret ranges (e.g. @auth/prisma-adapter, bcryptjs, zod, framer-motion, zustand, react-hook-form).
Expected: Reproducible installs. The lockfile covers this if CI installs with a frozen lockfile.
Do this: Deploy with `pnpm install --frozen-lockfile`. Optionally set `save-exact=true` in .npmrc.

2.4 `package.json:45`

Audit report — Visionia Electronics

Run #2105 · version 1.0.3 · B1072-24246 · M. Ismail H · 7 Nov 2026 13:45 UTC · engine claude-sonnet-5-5

Found: @types/bcryptjs ^2.4.6 is a stub types package, while bcryptjs ^3 ships its own types. The two versions are mismatched.
Expected: No redundant @types package for bcryptjs 3.x.
Do this: Run `pnpm remove @types/bcryptjs`.

2.5 `package.json:6`
Found: No audit step in scripts or CI, and the installed versions of next 16.4.3 and react 19.3.0 cannot be checked against advisories from the digest alone.
Expected: A routine vulnerability scan of the lockfile.
Do this: Run `pnpm audit --prod` before release and add it to CI.

2.6 `pnpm-lock.yaml`
Found: Lockfile is present (165 KB) and matches the declared packageManager pnpm@10.17.0. tsconfig.tsbuildinfo (190 KB) is also committed, which is a build artifact.
Expected: A committed lockfile (met). Build artifacts are not committed.
Do this: Add tsconfig.tsbuildinfo to .gitignore and run `git rm --cached tsconfig.tsbuildinfo`.

3 · Authorisation & access control

WARNING · 55/100

What we look for: All routes, controllers and API endpoints are traced. Each one that creates, updates or deletes is checked for an authentication guard; admin and management routes for a role check; lookups by id for an ownership check (can user A read user B's record by changing the id?).

Passwords hashed, but most admin/mutating guards could not be verified from the excerpts shown

Passwords are hashed with bcrypt and the NextAuth config looks sound, with a role stored in the JWT. However, the bodies of the admin server actions, cart actions, the order page and the payment callbacks were not shown, so no guard or ownership check could be confirmed. Several issues are visible: public registration with no rate limiting, a duplicate payment route tree, and unverified IPN handling.

3.1 `actions/admin.ts`
Found: Server actions for admin mutations (products, coupons, banners, orders, reviews) exist, but their contents were not provided. No role check is visible.
Expected: Each exported action calls auth(), rejects non-ADMIN roles, and validates input server-side.
Do this: Add a requireAdmin() helper at the top of every exported action in actions/admin.ts and lib/services/admin.ts that throws unless session.user.role === 'ADMIN'. Do not rely on layout-only checks.

3.2 `app/admin/layout.tsx`
Found: Admin pages are guarded (if at all) at the layout level. Layout guards do not protect server actions or API calls invoked directly.
Expected: Role check in the layout, in each page, and inside every server action and service function.
Do this: Enforce the role check in the data layer (lib/admin.ts, lib/services/admin.ts) as well as in the layout.

3.3 `app/orders/[orderId]/page.tsx`
Found: Order lookup by orderId in the URL. The ownership check could not be confirmed.
Expected: The query filters by both id and session.user.id, with an admin override, and returns 404 otherwise.
Do this: Use prisma.order.findFirst({ where: { id: orderId, userId: session.user.id } }) and redirect to /login when there is no session.

3.4 `actions/cart.ts`
Found: Cart mutations (add, update, remove) exist. It is unclear whether they are tied to the session user or accept a client-supplied cart or user id.
Expected: The cart is resolved from auth() server-side. Item ids are verified to belong to that cart.
Do this: Derive userId from auth(), never from the input. When updating or deleting a cart item, include cartId or userId in the where clause.

3.5 `app/api/payments/sslcommerz/ipn/route.ts`
Found: Public IPN and callback endpoints exist, duplicated in both /api/payment and /api/payments trees. The code that verifies them was not shown.
Expected: Callbacks revalidate the transaction with SSLCommerz's validation API server-side, check the amount and order, and are idempotent. Only one route tree exists.
Do this: Confirm the handler calls the validator and compares amount and tran_id against the order before marking it PAID. Remove the redundant route tree.

3.6 `actions/auth.ts:9`
Found: registerUser is a public server action with no rate limiting. It reveals whether an email is registered through its 'account already exists' error.
Expected: Rate limiting and a generic response to prevent account enumeration and abuse.
Do this: Add IP-based rate limiting, and consider email verification. Make the response uniform where practical.

3.7 `app/api/products/search/route.ts`
Found: Public search endpoint. It must not return unpublished products.
Expected: Filters on isPublished: true and isActive categories.
Do this: Ensure the query includes isPublished: true and returns only public fields.

Audit report — Visionia Electronics

Run #2105 · version 1.0.3 · B1072-24246 · M. Ismail H · 7 Nov 2026 13:45 UTC · engine claude-sonnet-5-5

3.8 `lib/auth.ts:89`
Found: The auth config is cast to any, and the role in the JWT is only refreshed at sign-in, so demotion or deletion of an admin does not take effect until the token expires.
Expected: Role re-checked against the DB for sensitive admin operations.
Do this: Re-read the user role from the DB inside requireAdmin(), or shorten the session maxAge.

4 · Database access rules

WARNING · 70/100

What we look for: Raw SQL and query builders for string-built statements; delete/update statements for missing WHERE; multi-tenant or per-user data for scoping; the schema script or migrations for completeness, foreign keys, indexes on foreign keys and sensible cascade rules.

Prisma ORM used with no visible injection paths, but most DB code was not shown, so the review is partial

The visible code uses Prisma with parameterised queries, bcrypt-hashed passwords and a cascade-aware schema, and no string-built SQL or unscoped delete appeared in the excerpts. The service files, admin actions, payment handlers and migrations were not included, so injection, scoping and N+1 risks there could not be verified. Duplicate payment route trees and a schema that is only partly visible are further concerns.

4.1 `lib/services/admin.ts`
Found: Admin service and actions contain the delete/update calls and any $queryRaw usage, but the contents were not in the digest.
Expected: All deletes/updates scoped by unique id or owner. No $queryRawUnsafe or string-interpolated SQL.
Do this: Audit every prisma.*.delete/update/deleteMany/updateMany and any $queryRaw for scoping. Use tagged-template $queryRaw with Prisma.sql parameters only.

4.2 `app/api/products/search/route.ts`
Found: Public search endpoint takes user input; its query implementation was not visible, so raw SQL/ILIKE concatenation could not be ruled out.
Expected: Search built with Prisma `contains` filters or parameterised tagged templates, with a length cap on input.
Do this: Use prisma.product.findMany({ where: { name: { contains: q, mode: 'insensitive' } }, take: N }). Never interpolate q into SQL text.

4.3 `lib/services/cart.ts`
Found: Cart, order and wishlist data is per-user. Scoping by session userId was not verifiable from the digest.
Expected: Every cart/order/wishlist query filters by the authenticated userId, never an id supplied by the client.
Do this: Derive userId from auth() on the server and include it in every where clause. On order pages, check order.userId === session.user.id before returning data.

4.4 `prisma/migrations/20261102023010_init/migration.sql`
Found: Migration timestamps are in the year 2026 and seven migrations exist, alongside database/schema.sql. Two competing schema sources can drift.
Expected: One authoritative schema path that builds from empty, with indexes on all foreign keys (Review.productId, Product.categoryId, Order.userId etc.).
Do this: Run `prisma migrate reset` on an empty DB to confirm it builds. Add @@index on every FK column that lacks one (e.g. Review.productId) and delete or sync database/schema.sql.

4.5 `app/api/payments/sslcommerz/ipn/route.ts`
Found: Duplicate route trees exist at /api/payment/ and /api/payments/ for SSLCommerz callbacks, doubling the number of paths that update order and payment rows.
Expected: A single callback route set, with payment updates scoped by transaction id and wrapped in a transaction for idempotency.
Do this: Remove the redundant tree and make the status updates idempotent: updateMany where { tranId, status: 'PENDING' } inside prisma.$transaction.

5 · Code structure & duplication

WARNING · 72/100

What we look for: Module boundaries, business logic leaking into controllers or views, oversized files and functions, duplicated blocks, dead and commented-out code, hard-coded values that should be configuration.

Duplicated SSLCommerz route trees and blanket lint suppressions make the payment flow harder to maintain

The payment callbacks exist twice under both /api/payment and /api/payments, which invites drift and confusion about which endpoint is live. Several files also disable type checking at the top, which hides problems in the auth and form code. The rest of the visible structure looks reasonable, though many files were truncated, so this score is provisional.

5.1 `app/api/payments/sslcommerz/ipn/route.ts`
Found: Full duplicate of the SSLCommerz route tree (cancel, fail, ipn, success) also present under app/api/payment/sslcommerz/.
Expected: A single canonical route tree for SSLCommerz callbacks, with the other path removed or reduced to a redirect.
Do this: Confirm which path the SSLCommerz dashboard and NEXT_PUBLIC_SITE_URL callbacks use, delete the other route folder, and keep the shared logic in lib/payments/sslcommerz-route-handlers.ts.

5.2 `app/api/payment/sslcommerz/ipn/route.ts`
Found: The same four callback handlers are duplicated between the singular and plural route folders.
Expected: Each callback is defined once and imported by any route that needs it.
Do this: Make each route.ts a thin re-export of the handler in lib/payments/sslcommerz-route-handlers.ts, then remove the duplicate folder.

Audit report — Visionia Electronics

Run #2105 · version 1.0.3 · B1072-24246 · M. Ismail H · 7 Nov 2026 13:45 UTC · engine claude-sonnet-5-5

5.3 `lib/auth.ts:1`
Found: File-level /* eslint-disable @typescript-eslint/no-explicit-any */ and NextAuth(authOptions as any) disable type checking for the whole auth module.
Expected: Typed NextAuth config with the authOptions object typed as NextAuthConfig, and suppressions limited to the lines that need them.
Do this: Type authOptions with NextAuthConfig from next-auth, remove the file-level disable, and drop the `as any` cast on NextAuth(...).

5.4 `components/auth/auth-form.tsx:1`
Found: File-level eslint-disable for no-explicit-any, with useForm<any> and zodResolver(schema as any).
Expected: Separate typed schemas for login and register, with a typed form value union.
Do this: Define a typed form value type per mode, pass it to useForm<T>, and remove the file-wide disable.

6 · Build reproducibility

WARNING · 72/100

What we look for: Solution/manifest references point at folders that exist; SDK and framework versions are consistent; build output and vendored folders are excluded; Docker/compose files are coherent; the setup document names prerequisites and versions and runs the steps in the right order.

Likely builds from lockfile, but setup docs and committed artifacts leave gaps

The project has a pnpm lockfile, a pinned packageManager, Prisma migrations and a patched dependency that is present in the archive, so a clean install is plausible. However, the setup docs are only partly visible and omit Node version, the migrate/seed order is unclear, and a build artifact is committed. Several version and config inconsistencies could trip a fresh machine.

6.1 `package.json:6`
Found: Scripts include db:push, db:generate and db:seed but no migrate/deploy script; build only runs prisma generate.
Expected: A documented, scripted path to apply the committed migrations (prisma migrate deploy) before start.
Do this: Add "db:migrate": "prisma migrate deploy" and document the order: install, configure .env, migrate deploy, seed, build, start. Avoid db:push in favour of the shipped migrations.

6.2 `package.json`
Found: No "engines" field and no .nvmrc; @types/node is ^20 while Next 16 and React 19.3 need a modern Node.
Expected: Declared Node version consistent with the framework and the @types/node major.
Do this: Add engines.node (e.g. >=20.9) and an .nvmrc; align @types/node with that major. Name the Node version in README/HowToSetup.md prerequisites.

6.3 `README.md:62`
Found: Prerequisites list Node.js, pnpm, Git with no versions; says to create .env by hand rather than copying .env.example.
Expected: Prerequisites with versions, and 'cp .env.example .env' as a step; steps in the right order including migrations and seed.
Do this: State Node and pnpm versions (pnpm 10.17), add the cp step, then migrate deploy, seed, build and start.

6.4 `tsconfig.tsbuildinfo`
Found: TypeScript incremental build cache (190 KB) is committed to the repo.
Expected: Build output excluded from the archive.
Do this: Delete the file and add *.tsbuildinfo to .gitignore.

6.5 `prisma/migrations/20261102023010_init/migration.sql`
Found: Migration folders are dated 2026 and database/schema.sql and database/seed.sql also exist alongside Prisma migrations.
Expected: One authoritative schema path; migration timestamps sensible.
Do this: Document which of the Prisma migrations or database/schema.sql is the source of truth, or remove the duplicate. Confirm migration ordering applies cleanly.

6.6 `app/api/payment/sslcommerz/ipn/route.ts`
Found: Duplicate route trees exist: app/api/payment/... and app/api/payments/... for the same SSLCommerz callbacks.
Expected: A single callback path, consistent with the docs and env configuration.
Do this: Keep one tree, or make one re-export the other, and document the callback URLs.

6.7 `.env.example:22`
Found: SSLCOMMERZ vars are marked optional, and NEXT_DEPLOYMENT_ID and VERCEL_DEPLOYMENT_ID are placeholders; README omits AUTH_URL/NEXT_PUBLIC_SITE_URL.
Expected: README and .env.example list the same variables, with required vs optional clearly marked.
Do this: Sync the README env block with .env.example and mark which variables are required to boot.

5 warnings will be shown publicly on the listing until a later version clears them.

AUDIT LOG

```
$ unpack Visionia-source.zip
  115 files, 2.0 MB, .tsx ×48, .ts ×35, .sql ×9, .json ×6, .md ×4, .mjs ×2
▸ probe: https://visionia-electronics.vercel.app/
```

Audit report — Visionia Electronics

Run #2105 · version 1.0.3 · B1072-24246 · M. Ismail H · 7 Nov 2026 13:45 UTC · engine claude-sonnet-5-5

```
  HTTP 200 in 1866 ms, final URL https://visionia-electronics.vercel.app/, title "Visionia | Premium
Electronics Accessories"
! secrets: 12 candidate line(s) flagged for review
▸ requirements: comparing the archive with your requirement list
▸ secrets: Secrets & credentials
▸ deps: Dependency vulnerabilities
▸ authz: Authorisation & access control
▸ dbrules: Database access rules
▸ structure: Code structure & duplication
▸ build: Build reproducibility
× R5 Authentication and roles on every mutating endpoint — partial
! R6 Pinned dependencies without critical CVEs — partial
× R8 Working live demo — partial
✓ requirements: 12/15 met · 0 must-have missing · 3 nice-to-have missing
✓ secrets: No real credentials found; only clear placeholders in env example and docs (93)
! structure: Duplicated SSLCommerz route trees and blanket lint suppressions make the payment flow harder to
maintain (72) · 4 finding(s)
! deps: Lockfile present, but beta auth, caret ranges and an unverifiable patched SDK reduce confidence (72) ·
6 finding(s)
! dbrules: Prisma ORM used with no visible injection paths, but most DB code was not shown, so the review is
partial (70) · 5 finding(s)
! build: Likely builds from lockfile, but setup docs and committed artifacts leave gaps (72) · 7 finding(s)
! authz: Passwords hashed, but most admin/mutating guards could not be verified from the excerpts shown (55) ·
8 finding(s)
  cache: 53,100 tokens read from cache, 21,240 written
— audit complete · 72/100 · 79,635 in / 9,200 out tokens
```